

판다스 (Pandas) (1)

2026-1학기

ICT융합학부 컴퓨터공학트랙

김대환

목차

1. Pandas 객체 소개
2. 데이터 인덱싱과 선택

Pandas 객체 소개

Pandas 객체 소개 (1)

- Pandas 객체

- 행과 열이 단순 정수형 인덱스가 아닌 레이블로 식별되는 NumPy의 구조화된 배열을 보강한 버전
- 3가지 자료 구조: **Series**, **DataFrame**, **Index**

- 판다스 임포트

```
import numpy as np
import pandas as pd
```

Pandas 객체 소개 (2) - Series

■ Pandas Series 객체

- 인덱싱된 데이터의 1차원 배열
- 리스트나 배열로부터 생성 가능
- 일련의 값과 인덱스를 모두 감싸고 있고, 각각 values와 index 속성으로 접근

```
data = pd.Series([0.25, 0.5, 0.75, 1.0])  
data
```

```
0    0.25  
1    0.50  
2    0.75  
3    1.00  
dtype: float64
```

```
data.values
```

```
array([ 0.25,  0.5 ,  0.75,  1.  ])
```

```
data.index
```

```
RangeIndex(start=0, stop=4, step=1)
```

```
data[1]
```

```
0.5
```

```
data[1:3]
```

```
1    0.50  
2    0.75  
dtype: float64
```

Pandas 객체 소개 (3) - Series

- **Series: 일반화된 NumPy 배열**

- 기본적으로 1차원 NumPy 배열과 호환 + **인덱스 존재**
- NumPy 배열에는 값에 접근하는 암묵적으로 정의된 정수형 인덱스 존재
- Series는 값에 연결된 명시적으로 정의된 인덱스 존재
 - 인덱스는 정수일 필요가 없고, 어떤 타입의 값으로도 구성 가능

```
data = pd.Series([0.25, 0.5, 0.75, 1.0],  
                  index=['a', 'b', 'c', 'd'])
```

data

```
a    0.25  
b    0.50  
c    0.75  
d    1.00  
dtype: float64
```

```
data['b']
```

0.5

```
data = pd.Series([0.25, 0.5, 0.75, 1.0],  
                  index=[2, 5, 3, 7])
```

data

```
2    0.25  
5    0.50  
3    0.75  
7    1.00  
dtype: float64
```

```
data[5]
```

0.5

Pandas 객체 소개 (4) - Series

- **Series: 특수화된 딕셔너리**

- 파이썬 딕셔너리: 일련의 임의의 값에 임의의 키를 매핑하는 구조
- Series: 타입이 지정된 키를 일련의 타입이 지정된 값에 매핑하는 구조
 - 타입이 지정되어, 특정 연산에서 딕셔너리보다 효율적임

```
population_dict = {'California': 38332521,  
                  'Texas': 26448193,  
                  'New York': 19651127,  
                  'Florida': 19552860,  
                  'Illinois': 12882135}  
population = pd.Series(population_dict)  
population
```

```
California    38332521  
Florida       19552860  
Illinois      12882135  
New York      19651127  
Texas         26448193  
dtype: int64
```

```
population['California']
```

```
38332521
```

```
population['California':'Illinois']
```

```
California    38332521  
Florida       19552860  
Illinois      12882135  
dtype: int64
```

Pandas 객체 소개 (5) - Series

Series 객체 구성하기

- Series 객체를 생성하는 형태 `pd.Series(data, index=index)`
- index는 선택 인수고, data는 많은 요소 중 하나

```
pd.Series([2, 4, 6])
```

```
0    2
1    4
2    6
dtype: int64
```

← data는 리스트나 NumPy 배열 가능,
그런 경우 index는 정수가 기본

```
pd.Series(5, index=[100, 200, 300])
```

```
100    5
200    5
300    5
dtype: int64
```

← data는 지정된 인덱스를
채우기 위한 스칼라 값

```
pd.Series({'2':'a', 1:'b', 3:'c'})
```

```
1    b
2    a
3    c
dtype: object
```

← data는 딕셔너리 가능, index를 딕셔너리 키를 사용

```
pd.Series({'2':'a', 1:'b', 3:'c'}, index=[3, 2])
```

```
3    c
2    a
dtype: object
```

← 인덱스를 명시적으로 설정 가능

Pandas 객체 소개 (6) – DataFrame

- DataFrame: 일반화된 NumPy 배열
 - 유연한 행 인덱스와 유연한 열 이름을 가진 2차원 배열
 - 정렬된 Series 객체의 연속

```
area_dict = {'California': 423967, 'Texas': 695662, 'New York': 141297,
             'Florida': 170312, 'Illinois': 149995}
area = pd.Series(area_dict)
area
```

```
California    423967
Florida       170312
Illinois      149995
New York      141297
Texas         695662
dtype: int64
```

```
states = pd.DataFrame({'population': population,
                       'area': area})
states
```

	area	population
California	423967	38332521
Florida	170312	19552860
Illinois	149995	12882135
New York	141297	19651127
Texas	695662	26448193

인덱스 속성 존재 →

```
states.index
```

```
Index(['California', 'Florida', 'Illinois', 'New York', 'Texas'], dtype='object')
```

```
states.columns
```

```
Index(['area', 'population'], dtype='object')
```

Pandas 객체 소개 (7) – DataFrame

- DataFrame: 특수한 딕셔너리

- DataFrame은 다양한 방법으로 구성 가능

- DataFrame 구성 방법들

- 단일 Series 객체에서 구성하기

```
pd.DataFrame(population, columns=['population'])
```

	population
California	38332521
Florida	19552860
Illinois	12882135
New York	19651127
Texas	26448193

- 딕셔너리의 리스트에서 구성하기

```
data = [{'a': i, 'b': 2 * i}  
        for i in range(3)]  
pd.DataFrame(data)
```

	a	b
0	0	0
1	1	2
2	2	4

```
pd.DataFrame([{'a': 1, 'b': 2}, {'b': 3, 'c': 4}])
```

	a	b	c
0	1.0	2	NaN
1	NaN	3	4.0

Pandas 객체 소개 (8) – DataFrame

- DataFrame 구성 방법들 (Cont')

- Series 객체의 딕셔너리에서 구성하기

```
pd.DataFrame({'population': population,  
              'area': area})
```

	area	population
California	423967	38332521
Florida	170312	19552860
Illinois	149995	12882135
New York	141297	19651127
Texas	695662	26448193

- 2차원 NumPy 배열에서 구성하기

```
pd.DataFrame(np.random.rand(3, 2),  
             columns=['foo', 'bar'],  
             index=['a', 'b', 'c'])
```

	foo	bar
a	0.865257	0.213169
b	0.442759	0.108267
c	0.047110	0.905718

- NumPy의 구조화된 배열에서 구성하기

```
A = np.zeros(3, dtype=[('A', 'i8'), ('B', 'f8')])  
A
```

```
array([(0, 0.0), (0, 0.0), (0, 0.0)],  
      dtype=[('A', '<i8'), ('B', '<f8')])
```

```
pd.DataFrame(A)
```

	A	B
0	0	0.0
1	0	0.0
2	0	0.0

Pandas 객체 소개 (9) – Index

- Index 객체

- Series와 DataFrame 객체가 데이터를 참조하고 수정하게 해주는 명시적인 인덱스
- 불변의 배열이나 정렬된 집합

```
ind = pd.Index([2, 3, 5, 7, 11])  
ind
```

```
Int64Index([2, 3, 5, 7, 11], dtype='int64')
```

← 정수 리스트로부터 Index를 구성

Pandas 객체 소개 (10) – Index

- Index : 불변의 배열
 - 여러 면에서 배열처럼 동작

```
ind[1]
```

```
3
```

← 표준 파이썬 인덱싱 표기법 사용 (슬라이스 가능)

```
ind[::2]
```

```
Int64Index([2, 5, 11], dtype='int64')
```

```
print(ind.size, ind.shape, ind.ndim, ind.dtype)
```

```
5 (5,) 1 int64
```

← NumPy와 비슷한 속성을 가짐

```
ind[1] = 0
```

← 일반적인 방법으로는 변경 불가

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-34-40e631c82e8a> in <module>()  
----> 1 ind[1] = 0
```

```
/Users/jakevdp/anaconda/lib/python3.5/site-packages/pandas/indexes/base.py
```

```
1243  
1244     def __setitem__(self, key, value):  
-> 1245         raise TypeError("Index does not support mutable operations")  
1246  
1247     def __getitem__(self, key):
```

```
TypeError: Index does not support mutable operations
```

Pandas 객체 소개 (11) – Index

- Index : 정렬된 집합
 - 합집합, 교집합, 차집합 계산 가능

```
indA = pd.Index([1, 3, 5, 7, 9])  
indB = pd.Index([2, 3, 5, 7, 11])
```

```
indA & indB  # intersection
```

```
Int64Index([3, 5, 7], dtype='int64')
```

```
indA | indB  # union
```

```
Int64Index([1, 2, 3, 5, 7, 9, 11], dtype='int64')
```

```
indA ^ indB  # symmetric difference
```

```
Int64Index([1, 2, 9, 11], dtype='int64')
```

데이터 인덱싱과 선택

데이터 인덱싱과 선택 (1) - Series

- Series에서 데이터 선택

- 여러 면에서 1차원 NumPy 배열과 표준 파이썬 딕셔너리처럼 동작

- Series: 딕셔너리

- (딕셔너리와 마찬가지로) 키의 집합을 값의 집합에 매핑

```
import pandas as pd
data = pd.Series([0.25, 0.5, 0.75, 1.0],
                  index=['a', 'b', 'c', 'd'])
data
```

```
a    0.25
b    0.50
c    0.75
d    1.00
dtype: float64
```

```
data['b']
```

```
0.5
```


데이터 인덱싱과 선택 (2) - Series

- Series: 딕셔너리 (Cont')

- 키/인덱스와 값을 조사하기 위해 딕셔너리와 유사한 파이썬 표현식/메소드 사용

```
'a' in data
```

```
True
```

```
data.keys()
```

```
Index(['a', 'b', 'c', 'd'], dtype='object')
```

```
list(data.items())
```

```
[('a', 0.25), ('b', 0.5), ('c', 0.75), ('d', 1.0)]
```

- 새로운 인덱스 값에 할당하여, Series를 확장 가능

```
data['e'] = 1.25
```

```
data
```

```
a    0.25
```

```
b    0.50
```

```
c    0.75
```

```
d    1.00
```

```
e    1.25
```

```
dtype: float64
```

데이터 인덱싱과 선택 (3) - Series

- Series: 1차원 배열

- 슬라이스, 마스킹, 팬시 인덱싱 등 NumPy 배열과 똑같은 기본 메커니즘으로 배열 형태의 아이টে을 선택 가능

```
# slicing by explicit index  
data['a':'c']
```

```
a    0.25  
b    0.50  
c    0.75  
dtype: float64
```

```
# slicing by implicit integer index  
data[0:2]
```

```
a    0.25  
b    0.50  
dtype: float64
```

```
# masking  
data[(data > 0.3) & (data < 0.8)]
```

```
b    0.50  
c    0.75  
dtype: float64
```

```
# fancy indexing  
data[['a', 'e']]
```

```
a    0.25  
e    1.25  
dtype: float64
```

← data['a':'c']는 최종 인덱스를 포함하지만,
data[0:2]는 최종 인덱스가 제외

데이터 인덱싱과 선택 (4) - Series

- 인덱서: loc와 iloc

- 슬라이싱과 인덱싱의 관례적 표기법은 혼동 일으킴

- (예) Series가 명시적인 정수 인덱스를 가지고 있다면

→ data[1]과 같은 인덱싱 연산은 명시적인 인덱스를 사용하겠지만,
data[1:3] 같은 슬라이싱 연산은 파이썬의 암묵적 인덱스를 사용

```
data = pd.Series(['a', 'b', 'c'], index=[1, 3, 5])  
data
```

```
1    a  
3    b  
5    c  
dtype: object
```

```
# explicit index when indexing  
data[1]
```

```
'a'
```

```
# implicit index when slicing  
data[1:3]
```

```
3    b  
5    c  
dtype: object
```

데이터 인덱싱과 선택 (5) - Series

■ 인덱서: loc와 iloc (Cont')

- 인덱싱 방식을 명시적으로 드러내는 특별한 인덱서(Indexer) 속성을 제공
- loc 속성: 명시적인 인덱스를 참조하는 인덱싱과 슬라이싱
- iloc 속성: 암묵적인 스타일의 인덱싱과 슬라이싱

```
data = pd.Series(['a', 'b', 'c'], index=[1, 3, 5])  
data
```

```
1    a  
3    b  
5    c  
dtype: object
```

```
data.loc[1]
```

```
'a'
```

```
data.iloc[1]
```

```
'b'
```

```
data.loc[1:3]
```

```
1    a  
3    b  
dtype: object
```

VS

```
data.iloc[1:3]
```

```
3    b  
5    c  
dtype: object
```

데이터 인덱싱과 선택 (6) - DataFrame

■ DataFrame: 딕셔너리

■ 관련 Series 객체의 딕셔너리

```
area = pd.Series({'California': 423967, 'Texas': 695662,
                  'New York': 141297, 'Florida': 170312,
                  'Illinois': 149995})
pop = pd.Series({'California': 38332521, 'Texas': 26448193,
                 'New York': 19651127, 'Florida': 19552860,
                 'Illinois': 12882135})
data = pd.DataFrame({'area':area, 'pop':pop})
data
```

	area	pop
California	423967	38332521
Florida	170312	19552860
Illinois	149995	12882135
New York	141297	19651127
Texas	695662	26448193

```
data['density'] = data['pop'] / data['area']
data
```

	area	pop	density
California	423967	38332521	90.413926
Florida	170312	19552860	114.806121
Illinois	149995	12882135	85.883763
New York	141297	19651127	139.076746
Texas	695662	26448193	38.018740

```
data['area']
```

← 딕셔너리 스타일 인덱싱

```
California    423967
Florida       170312
Illinois      149995
New York      141297
Texas         695662
Name: area, dtype: int64
```

```
data.area
```

← 속성 스타일로 접근 가능

```
California    423967
Florida       170312
Illinois      149995
New York      141297
Texas         695662
Name: area, dtype: int64
```

```
data.area is data['area']
```

```
True
```

```
data.pop is data['pop']
```

← pop 메소드 존재에는 혼동

```
False
```

데이터 인덱싱과 선택 (7) - DataFrame

- DataFrame: 2차원 배열
 - values 속성을 이용하여 데이터 배열 확인 가능

```
data.values
```

```
array([[ 4.23967000e+05,  3.83325210e+07,  9.04139261e+01],
       [ 1.70312000e+05,  1.95528600e+07,  1.14806121e+02],
       [ 1.49995000e+05,  1.28821350e+07,  8.58837628e+01],
       [ 1.41297000e+05,  1.96511270e+07,  1.39076746e+02],
       [ 6.95662000e+05,  2.64481930e+07,  3.80187404e+01]])
```

```
data.T
```

	California	Florida	Illinois	New York	Texas
area	4.239670e+05	1.703120e+05	1.499950e+05	1.412970e+05	6.956620e+05
pop	3.833252e+07	1.955286e+07	1.288214e+07	1.965113e+07	2.644819e+07
density	9.041393e+01	1.148061e+02	8.588376e+01	1.390767e+02	3.801874e+01

← 행과 열 변경

```
data.values[0]
```

```
array([ 4.23967000e+05,  3.83325210e+07,  9.04139261e+01])
```

데이터 인덱싱과 선택 (8) - DataFrame

■ DataFrame: 2차원 배열 (Cont')

- 단일 인덱스를 전달하면 열에 접근 가능

```
data['area']
```

```
California    423967  
Florida       170312  
Illinois      149995  
New York      141297  
Texas         695662  
Name: area, dtype: int64
```

- 인덱서 사용 접근

```
data.iloc[:3, :2]
```

	area	pop
California	423967	38332521
Florida	170312	19552860
Illinois	149995	12882135

```
data.loc[:, 'Illinois', : 'pop']
```

	area	pop
California	423967	38332521
Florida	170312	19552860
Illinois	149995	12882135

```
data.loc[data.density > 100, ['pop', 'density']]
```

	pop	density
Florida	19552860	114.806121
New York	19651127	139.076746

```
data.iloc[0, 2] = 90  
data
```

	area	pop	density
California	423967	38332521	90.000000
Florida	170312	19552860	114.806121
Illinois	149995	12882135	85.883763
New York	141297	19651127	139.076746
Texas	695662	26448193	38.018740

데이터 인덱싱과 선택 (9) - DataFrame

- 추가적인 인덱싱 규칙
 - 인덱싱은 열을 참조하는 반면, 슬라이싱은 행을 참조

```
data['Florida':'Illinois']
```

	area	pop	density
Florida	170312	19552860	114.806121
Illinois	149995	12882135	85.883763

```
data[1:3]
```

	area	pop	density
Florida	170312	19552860	114.806121
Illinois	149995	12882135	85.883763

```
data[data.density > 100]
```

	area	pop	density
Florida	170312	19552860	114.806121
New York	141297	19651127	139.076746

← 마스킹 연산은 행 단위로 해석

Q & A
